

모델 컨텍스트 프로토콜 (MCP): 환경, 보안 위협 및 향후 연구 방향

Xinyi Hou, Yanjie Zhao, Snenao Wang, Haoyu Wang. 2025년..모델 컨텍스트 프로토콜 (MCP): 환경, 보안 위협, 그리고 미래 연구 방향..1, 1 (2025년 4월), 20페이지..<https://doi.org/10.1145/nnnnnnnn..nnnnnnnn>

이 논문은 AI 생태계에서 MCP의 역할과 중요성을 강조하며, MCP의 구성 요소, 워크플로우, 그리고 MCP 서버의 라이프사이클(생성, 운영, 업데이트)에 초점을 맞춰 MCP에 대한 포괄적인 개요를 제공합니다. 또한 각 단계와 관련된 보안 및 개인 정보 보호 위험을 분석하고 잠재적인 위협을 완화하기 위한 전략을 제시합니다. 본 논문은 또한 업계 리더들의 채택 및 다양한 활용 사례를 포함하여 현재 MCP 환경을 검토하고, MCP의 통합을 지원하는 도구 및 플랫폼을 살펴봅니다. 마지막으로, AI 생태계가 계속 발전함에 따라 MCP의 안전하고 지속 가능한 개발을 보장하기 위한 이해 관계자를 위한 권장 사항을 제시합니다.

CCS 개념: << 일반 및 참고 - 설문 및 개요; + 보안 및 개인 정보 보호 - 소프트웨어 및 애플리케이션 보안; << 컴퓨팅 방법론 - 인공지능.

추가 주요 단어 및 구: 모델 컨텍스트 프로토콜 (MCP), 비전 페이퍼, 보안

ACM 참조 형식:

Xinyi Hou, Yanjie Zhao, Snenao Wang, Haoyu Wang. 2025년..모델 컨텍스트 프로토콜 (MCP): 환경, 보안 위협, 그리고 미래 연구 방향..1, 1 (2025년 4월), 20페이지..<https://doi.org/10.1145/nnnnnnnn..nnnnnnnn>

1. 서론

최근 몇 년 동안, 광범위한 도구 및 데이터 소스와 상호 작용할 수 있는 자율 AI 에이전트에 대한 비전이 상당한 주목을 받고 있습니다. 이러한 발전은 2023년에 OpenAI의 기능 호출 기능을 통해 가속화되었으며, 이를 통해 언어 모델이 구조적인 방식으로 외부 API를 호출할 수 있게 되었습니다 [38]. 이러한 발전은 LLM의 기능을 확장하여 실시간 데이터 검색, 계산 수행, 외부 시스템과의 상호 작용을 가능하게 했습니다. 기능 호출이 보급됨에 따라, 이를 중심으로 생태계가 형성되었습니다. OpenAI는 ChatGPT 플러그인을 도입하여 개발자가 ChatGPT를 위한 호출 가능한 도구를 구축할 수 있도록 했습니다. Coze [4] 및 Yuangi [50]과 같은 LLM 앱 스토어는 특정 도구를 지원하는 플러그인 스토어를 출시했습니다.

해당 저자: haoyuwang@hust.ed

저자 정보: 신이 후 (xinyihou@hust.edu.cn), 후중대학교, 중국 화청; 야지에 저우 (yanjie_zhao@hust.edu.cn), 후중대학교, 중국 화청; 세나오 왕 (shenaowang @hust.edu.cn), 후중대학교, 중국 화청; 하오유 왕 (haoyuwang@hust.edu.cn), 후중대학교, 중국 화청.

본 자료의 전부 또는 일부를 개인적인 용도나 교육 목적으로 디지털 또는 인쇄본으로 복제하는 데 대한 허가가, 복제가 이익을 목적으로 또는 상업적 이점을 얻기 위해 제작되거나 배포되지 않는 경우, 비용 없이 부여됩니다. 복제본에는 본 내용과 첫 페이지에 전체 출처 정보를 포함해야 합니다. 저자가 아닌 다른 소유자의 자료에 대한 저작권은 존중되어야 합니다. 출처를 명시한 경우 복제는 허용됩니다. 그렇지 않은 경우, 또는 재게시, 서버에 게시, 목록으로 재배포하는 경우에는 사전적인 허가가 필요합니다. permissions@acm.org에서 허가를 요청하십시오. © 2025 저작권은 소유자/저자에게 있습니다. ACM에 게재 권한이 부여됩니다. ACM XXXX-XXXX/2025/4-ART <https://doi.org/10.1145/nnnnnnnn..nnnnnnnn>

그들의 플랫폼... LangChain [26] 및 LlamaIndex [29]과 같은 프레임워크는 표준화된 도구 인터페이스를 제공하여 LLM을 외부 서비스와 통합하는 것을 더 쉽게 만들어 줍니다. Anthropic, Google, Meta를 포함한 다른 AI 제공업체들도 유사한 메커니즘을 도입하여 채택을 더욱 촉진했습니다. 이러한 발전에도 불구하고, 도구 통합은 여전히 분산되어 있습니다. 개발자는 각 서비스에 대해 인터페이스를 수동으로 정의하고, 인증을 관리하고, 실행 로직을 처리해야 합니다. 플랫폼 간에 기능 호출 메커니즘이 다르기 때문에 중복된 구현이 필요합니다. 또한, 현재 접근 방식은 미리 정의된 워크플로우에 의존하기 때문에 AI 에이전트가 도구를 동적으로 발견하고 조율하는 유연성이 제한됩니다.

2024년 말, Anthropic은 Model Context Protocol (MCP)[3]를 도입했습니다. 이는 AI 도구와의 상호 작용을 표준화하는 일반적인 프로토콜입니다. 언어 서버 프로토콜 (LSP)[22]에서 영감을 얻은 MCP는 AI 애플리케이션이 외부 도구와 동적으로 소통할 수 있는 유연한 프레임워크를 제공합니다. 사전 정의된 도구 매핑에 의존하는 대신, MCP는 AI 에이전트가 작업 맥락에 따라 도구를 자율적으로 검색, 선택, 그리고 조정할 수 있도록 합니다. 또한, MCP는 사용자가 필요에 따라 데이터를 주입하거나 작업을 승인하는 인간-AI 협업 메커니즘도 지원합니다. MCP는 인터페이스를 통합하여 AI 애플리케이션의 개발을 단순화하고, 복잡한 워크플로우를 처리하는 유연성을 향상시킵니다. 출시 이후, MCP는 특정 프로토콜에서 빠르게 핵심적인 기반 기술로 자리 잡았습니다. GitHub [41], Slack [42], 심지어 블렌더 [1]와 같은 3D 디자인 도구를 포함한 다양한 시스템에서 모델에 액세스할 수 있는 수천 개의 커뮤니티 기반 MCP 서버가 등장했습니다. Cursor [12] 및 Claude Desktop [2]와 같은 도구는 새로운 서버를 설치하여 개발 도구, 생산성 플랫폼, 창의적인 환경을 모두 다중 모드 AI 에이전트로 확장할 수 있음을 보여줍니다.

MCP의 빠른 확산에도 불구하고, 보안, 도구 검색 가능성, 원격 배포 등 핵심 영역에서 포괄적인 해결책이 부족한 상태로, 생태계는 아직 초기 단계에 머물러 있습니다. 이러한 문제점은 추가적인 연구 및 개발의 기회를 제공합니다. MCP는 산업 분야에서 잠재력을 인정받고 있지만, 학술 연구에서는 아직까지 심층적인 분석이 이루어지지 않았습니다. 이러한 연구의 부족은 본 논문을 통해 MCP 생태계에 대한 최초의 분석을 제공하고, 아키텍처 및 워크플로우, MCP 서버의 라이프사이클 정의, 설치 파일 위조 및 도구 이름 충돌 등 각 단계별 잠재적인 보안 위험을 식별하는 것을 포함합니다. 이를 통해 본 연구는 MCP의 현재 상황에 대한 심층적인 분석을 제시하고, 핵심적인 함의를 강조하며, 미래 연구 방향을 제시하고, 지속 가능한 성장을 보장하기 위한 해결해야 할 과제를 제시합니다.

저희의 기여는 다음과 같습니다:

- (1) 우리는 MCP 생태계에 대한 최초의 분석을 제공하며, 그 구조, 구성 요소 및 워크플로우를 상세히 설명합니다.
- (2) 우리는 MCP 서버의 주요 구성 요소를 식별하고, 생성, 운영, 업데이트 단계 등 각 단계의 라이프사이클을 정의합니다. 또한, 각 단계와 관련된 잠재적인 보안 위험을 강조하고, AI와 도구 간의 상호 작용을 보호하는 방법에 대한 통찰력을 제공합니다.
- (3) 현재 MCP 생태계의 현황을 분석하고, 다양한 산업 및 플랫폼에서 채택, 다양성, 활용 사례 등을 조사합니다.
- (4) 우리는 MCP의 빠른 도입에 따른 영향을 논의하고, 이해 관계자들에게 나타나는 주요 과제를 식별하며, 보안, 확장성 및 거버넌스에 대한 미래 연구 방향을 제시하여 지속 가능한 성장을 보장합니다.

본 논문의 나머지 부분은 다음과 같이 구성되어 있습니다. § 2에서는 MCP 사용 여부에 따라 도구 호출 방식을 비교하며, 이 연구의 동기를 강조합니다. § 3에서는 MCP의 아키텍처를 설명하며, MCP 호스트, 클라이언트, 서버의 역할뿐만 아니라 MCP 서버의 수명 주기를 상세히 설명합니다. § 4에서는 주요 산업 참여자와 채택 트렌스를 중심으로 현재 MCP 환경을 분석합니다. § 5에서는 MCP 서버의 수명 주기 전반에 걸쳐 발생하는 보안 및 개인 정보 관련 위험을 분석하고 완화 방안을 제시합니다.

§ 6에서는 동적 AI 환경에서 MCP의 확장성과 보안을 강화하기 위한 합의, 미래 과제, 그리고 제안 사항을 살펴봅니다. § 7에서는 LLM 애플리케이션에서의 도구 통합 및 보안과 관련된 기존 연구를 검토합니다. 마지막으로 § 8에서는 전체 논문을 마무리합니다.

2. 배경 및 동기

2.1. 알 도구

MCP 도입 이전, AI 애플리케이션은 수동 API 연결, 플러그인 기반 인터페이스, 에이전트 프레임워크 등 다양한 방법을 사용하여 외부 도구와 상호 작용했습니다. 그림 1에서 볼 수 있듯이, 이러한 접근 방식은 각 외부 서비스에 대해 특정 API를 통합해야 했기 때문에 복잡성이 증가하고 확장성이 제한되었습니다. MCP는 표준화된 프로토콜을 제공하여 여러 도구와의 원활하고 유연한 상호 작용을 가능하게 하여 이러한 문제를 해결합니다.

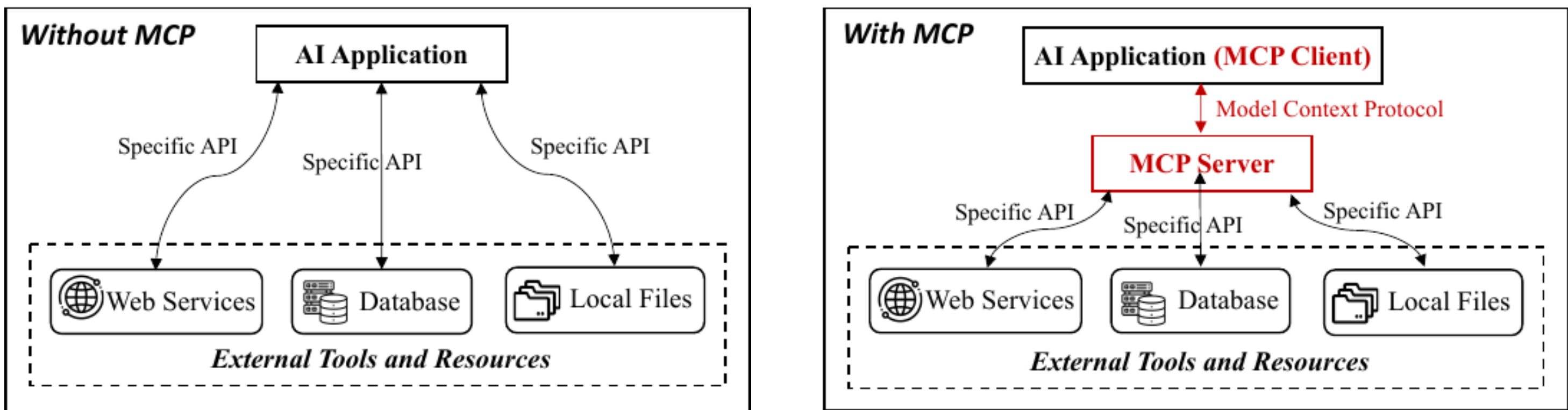


그림 1: MCP 유무에 따른 도구 호출 방식

2.1.1. 수동 API 연결... 기존 구현 방식에서는 AI 애플리케이션이 상호 작용하는 각 도구나 서비스에 대해 개발자가 수동으로 API 연결을 설정해야 했습니다. 이러한 과정에는 각 통합에 대한 사용자 정의 인증, 데이터 변환, 오류 처리 등이 필요했습니다. API 수가 증가함에 따라 유지 관리 부담이 커졌고, 이는 사용자 정의 API 연결 없이도 다양한 도구와 동적으로 연결할 수 있는, 확장하거나 수정하기 어려운 복잡한 시스템으로 이어지는 경우가 많았습니다. MCP는 이러한 복잡성을 제거하여 AI 모델이 사용자 정의 API 연결 없이도 다양한 도구와 동적으로 연결할 수 있도록 합니다.

2.1.2 표준화된 플러그인 인터페이스...수동적인 연결의 복잡성을 줄이기 위해, 2023년 11월에 도입된 OpenAI ChatGPT 플러그인과 같은 플러그인 기반 인터페이스는 AI 모델이 OpenAPI와 같은 표준화된 API 스키마를 통해 외부 도구와 연결할 수 있도록 했습니다. 예를 들어, OpenAI 플러그인 생태계에서는 Zapier와 같은 플러그인을 통해 모델이 미리 정의된 작업을 수행하도록 할 수 있습니다. 예를 들어, 이메일 전송 또는 CRM 기록 업데이트와 같은 작업을 수행할 수 있습니다. 그러나 이러한 상호 작용은 종종 단방향적이었으며, 작업 내 여러 단계를 유지하거나 조정하는 데 어려움을 겪었습니다. 또한, ByteDance Coze [4] 및 Tencent Yuanqi [50]과 같은 새로운 LLM 앱 스토어도 웹 서비스용 플러그인 스토어를 제공하고 있습니다. 이러한 플랫폼은 사용 가능한 도구 옵션을 확장했지만, 플러그인이 플랫폼에 종속되어 있어, 플랫폼 간 호환성을 제한하고 중복된 유지 관리 노력을 필요로 하는 고립된 생태계를 만들었습니다. MCP는 오픈 소스이며 플랫폼에 독립적이라는 점에서 두드러집니다. 이를 통해 AI 애플리케이션은 외부 도구와 풍부하고 양방향적인 상호 작용을 수행할 수 있으며, 복잡한 워크플로우를 용이하게 합니다.

2.1.3 AI 에이전트 도구 통합... LangChain [26]과 같은 AI 에이전트 프레임워크 및 유사한 도구 오케스트레이션 프레임워크의 등장으로, 모델이 미리 정의된 인터페이스를 통해 외부 도구를 호출하는 구조적인 방법을 제공하여 자동화 및 적응성을 향상시켰습니다 [55]. 그러나 이러한 도구의 통합 및 관리는 여전히 대부분 수동적인 작업이 필요했습니다.

도구가 증가함에 따라 복잡성이 증가했습니다. MCP는 표준화된 프로토콜을 제공하여 AI 에이전트가 통합 인터페이스를 통해 여러 도구를 원활하게 호출, 상호 작용, 연결할 수 있도록 하여, 이 과정을 단순화합니다. 이는 수동 설정의 필요성을 줄이고, 작업의 유연성을 향상시켜 에이전트가 광범위한 사용자 정의 통합 없이 복잡한 작업을 수행할 수 있도록 합니다.

2.1.4 RAG(Retrieval-Augmented Generation) 및 벡터 데이터베이스 RAG와 같은 문맥 정보를 검색하는 방법은 벡터 기반 검색을 활용하여 데이터베이스 또는 지식 베이스에서 관련 정보를 검색하고, 모델이 최신 정보로 응답을 보완할 수 있도록 합니다 [11, 16]. 이 접근 방식은 지식 부족 문제를 해결하고 모델의 정확도를 향상시켰지만, 정보 검색에만 국한되었습니다. 모델이 데이터를 수정하거나 워크플로우를 트리거하는 등의 능동적인 작업을 수행할 수 없었습니다. 예를 들어, RAG 기반 시스템은 고객 지원 AI가 제품 문서 데이터베이스에서 관련 섹션을 검색하여 도움을 줄 수 있습니다. 그러나 AI가 고객 기록을 업데이트하거나 문제를 인력 지원으로 이행해야 하는 경우, 텍스트 기반 응답을 제공하는 것 외에는 어떤 조치를 취할 수 없습니다. MCP는 텍스트 기반 정보 검색을 넘어, AI 모델이 외부 데이터 소스와 도구와 능동적으로 상호 작용할 수 있도록 하여, 통합된 워크플로우에서 검색 및 조치를 모두 지원합니다.

2.2. 동기

MCP는 AI 모델이 외부 도구와 상호 작용하고, 데이터를 가져오고, 작업을 실행하는 방식을 표준화할 수 있는 능력 덕분에 AI 커뮤니티에서 빠르게 확산되고 있습니다. 수동 API 연결, 플러그인 인터페이스, 에이전트 프레임워크의 한계를 해결함으로써, MCP는 AI와 도구 간의 상호 작용을 재정의하고 더욱 자율적이고 지능적인 에이전트 워크플로우를 가능하게 할 잠재력을 가지고 있습니다. MCP는 빠르게 확산되고 있으며, 유망한 잠재력을 가지고 있지만, 아직 초기 단계에 있으며, 여전히 불완전한 생태계를 가지고 있습니다. 보안 및 도구 검색과 같은 중요한 측면이 아직 완전히 해결되지 않았으며, 이는 미래 연구 및 개선의 여지를 남깁니다. 또한, MCP는 산업계에서 빠르게 확산되고 있지만, 학계에서는 아직 광범위하게 연구되지 않고 있습니다.

이러한 격차를 바탕으로, 본 연구는 현재 MCP 환경을 분석하고, 등장하는 생태계를 조사하며, 잠재적인 보안 위험을 식별하는 최초의 연구입니다. 또한, MCP의 장기적인 성공을 위한 핵심 과제를 제시하고, 이를 지원하기 위한 비전을 제시합니다.

3. MCP 아키텍처

3.1 핵심 구성 요소

MCP 아키텍처는 세 가지 핵심 구성 요소로 구성됩니다: MCP 호스트, MCP 클라이언트 및 MCP 서버. 이러한 구성 요소는 AI 애플리케이션, 외부 도구 및 데이터 소스 간의 원활한 통신을 가능하게 하며, 운영이 안전하고 적절하게 관리되도록 보장합니다. 그림 2에서 보듯이, 일반적인 워크플로우에서 사용자는 MCP 클라이언트로 프롬프트를 전송하고, MCP 서버를 통해 적절한 도구를 선택하여 필요한 정보를 검색하고 처리한 후, 결과를 사용자에게 알립니다.

3.1.1. MCP 호스트: MCP 호스트는 MCP 클라이언트를 실행하면서 AI 기반 작업을 수행할 수 있는 환경을 제공하는 AI 애플리케이션입니다. 외부 서비스와의 원활한 소통을 위해 상호 작용 도구 및 데이터를 통합합니다. 예를 들어, Claude Desktop (AI 기반 콘텐츠 제작), Cursor (AI 기반 IDE, 코드 완성 및 소프트웨어 개발), 그리고 복잡한 작업을 수행하는 자율 시스템으로 작동하는 AI 에이전트 등이 있습니다. MCP 호스트는 MCP 클라이언트를 호스팅하고 외부 MCP 서버와의 통신을 보장합니다.

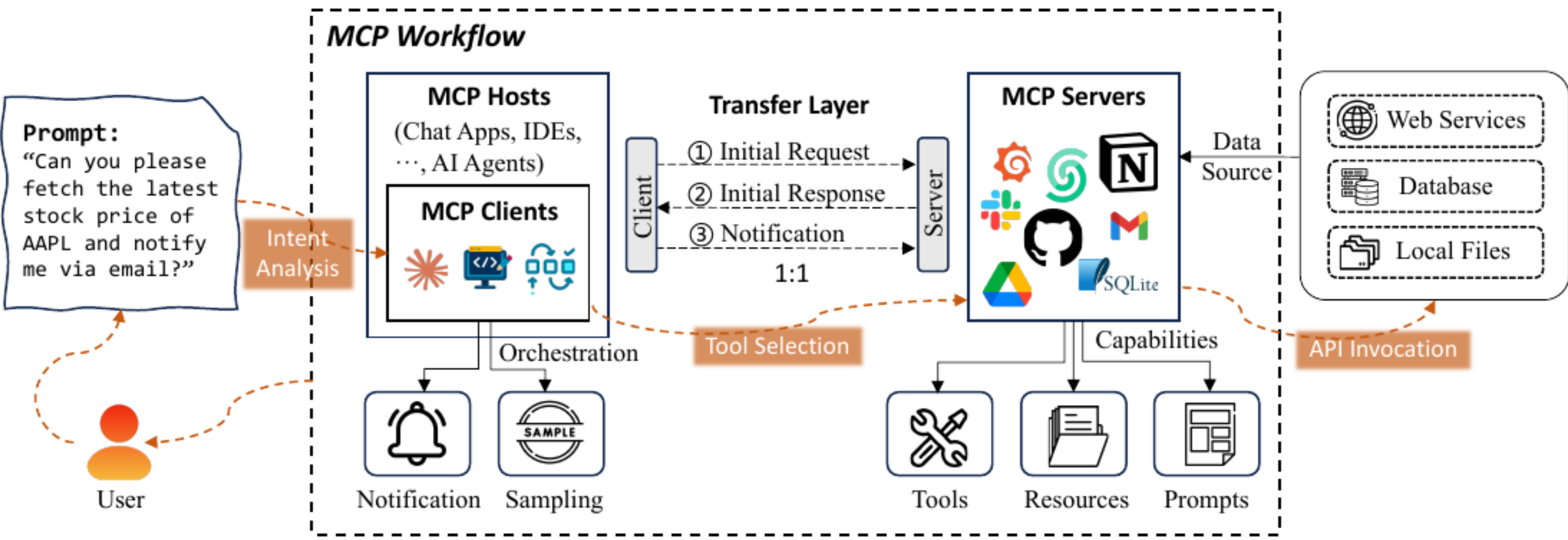


그림 2: MCP의 워크플로우

3.1.2 MCP 클라이언트: MCP 클라이언트는 호스트 환경 내에서 MCP 호스트와 하나 이상의 MCP 서버 간의 통신을 관리하는 역할을 수행합니다. 서버의 기능을 쿼리하고, 서버의 기능을 설명하는 응답을 검색하는 요청을 MCP 서버에 전송합니다. 이를 통해 호스트와 외부 도구 간의 원활한 상호 작용을 보장합니다. 요청 및 응답 관리를 뿐만 아니라, MCP 서버로부터 작업 진행 상황 및 시스템 상태에 대한 실시간 업데이트를 제공합니다. 또한 도구 사용 및 성능에 대한 데이터를 수집하여 최적화 및 정보에 입각한 의사 결정을 가능하게 합니다. MCP 클라이언트는 MCP 서버와 전송 레이어를 통해 통신하며, 호스트와 외부 리소스 간의 안전하고 안정적인 데이터 교환 및 원활한 상호 작용을 지원합니다.

3.1.3. MCP 서버: MCP 서버는 MCP 호스트 및 클라이언트가 외부 시스템에 접근하고 작업을 수행할 수 있도록 하며, 다음과 같은 세 가지 핵심 기능을 제공합니다: 도구, 리소스, 및 프롬프트.

도구: 외부 운영 지원 기능..MCP 서버가 AI 모델을 대신하여 외부 서비스 및 API를 호출하여 작업을 수행할 수 있도록 합니다. 클라이언트가 작업을 요청하면, MCP 서버는 적절한 도구를 식별하고, 해당 서비스와 상호 작용하여 결과를 반환합니다. 예를 들어, AI 모델이 실시간 날씨 데이터나 감성 분석을 필요로 하는 경우, MCP 서버는 관련 API에 연결하여 데이터를 가져오고, 이를 호스트에 전달합니다. 기존의 함수 호출 방식과 달리, MCP 서버의 도구는 모델이 상황에 따라 자동으로 적절한 도구를 선택하고 호출할 수 있도록 하여, 이 과정을 간소화합니다. 이러한 도구는 설정 후 표준화된 공급-소비 모델을 따르므로, 모듈화되어 재사용 가능하며 다른 애플리케이션에도 쉽게 접근할 수 있어, 시스템 효율성과 유연성을 향상시킵니다.

자원: AI 모델에 데이터를 노출하는 기능. 이 기능은 MCP 서버가 AI 모델에 제공할 수 있는 구조화 및 비구조화된 데이터에 대한 접근 권한을 제공합니다. 이러한 데이터는 로컬 저장소, 데이터베이스 또는 클라우드 플랫폼에서 가져올 수 있습니다. AI 모델이 특정 데이터를 요청하면, MCP 서버는 관련 정보를 검색하고 처리하여, 모델이 데이터 기반의 의사 결정을 내릴 수 있도록 합니다. 예를 들어, 추천 시스템은 고객 상호 작용 로그에 접근하거나, 문서 요약 작업은 텍스트 저장소에 쿼리를 실행할 수 있습니다.

• 템플릿: 워크플로우 최적화를 위한 재사용 가능한 템플릿. 이 템플릿과 워크플로우는 MCP 서버가 생성하고 유지하여 AI 응답을 최적화하고 반복적인 작업을 간소화합니다. 이를 통해 일관성 있는 응답을 제공하고 작업 효율성을 향상시킬 수 있습니다. 예를 들어, 고객 지원 챗봇은 이러한 템플릿을 사용하여 일관된

정확하고 신뢰할 수 있는 답변을 제공하는 동시에, 데이터 라벨링의 일관성을 유지하기 위해 미리 정의된 프롬프트를 활용할 수 있습니다.

3.2. 전송 계층 및 통신

전송 계층은 안전하고 양방향 통신을 보장하여 호스트 환경과 외부 시스템 간의 실시간 상호 작용 및 효율적인 데이터 교환을 가능하게 합니다. 전송 계층은 클라이언트의 초기 요청 전송, 서버의 응답(사용 가능한 기능 상세) 전달, 그리고 클라이언트를 지속적으로 업데이트 정보로 알리는 알림 교환을 관리합니다. MCP 클라이언트와 MCP 서버 간의 통신은 클라이언트가 서버의 기능을 쿼리하는 초기 요청부터 시작하는 체계적인 프로세스를 따릅니다. 요청을 수신하면, 서버는 사용 가능한 도구, 리소스, 그리고 클라이언트가 활용할 수 있는 프롬프트를 포함한 초기 응답을 제공합니다. 연결이 설정되면, 시스템은 서버 상태 또는 업데이트의 변경 사항이 실시간으로 클라이언트에 다시 전달되도록 지속적인 알림 교환을 유지합니다. 이러한 체계적인 통신은 고성능 상호 작용을 보장하고, AI 모델을 외부 리소스와 동기화하여 AI 애플리케이션의 효과를 향상시킵니다.

3.3. MCP 서버의 수명 주기

그림 3에 나타난 MCP 서버의 수명 주기는 크게 세 가지 단계로 구성됩니다. 즉, 생성, 운영, 업데이트 단계입니다. 각 단계는 MCP 서버의 안전하고 효율적인 작동을 보장하는 중요한 활동을 정의하며, 이를 통해 AI 모델과 외부 도구, 리소스, 프롬프트 간의 원활한 상호 작용을 가능하게 합니다.

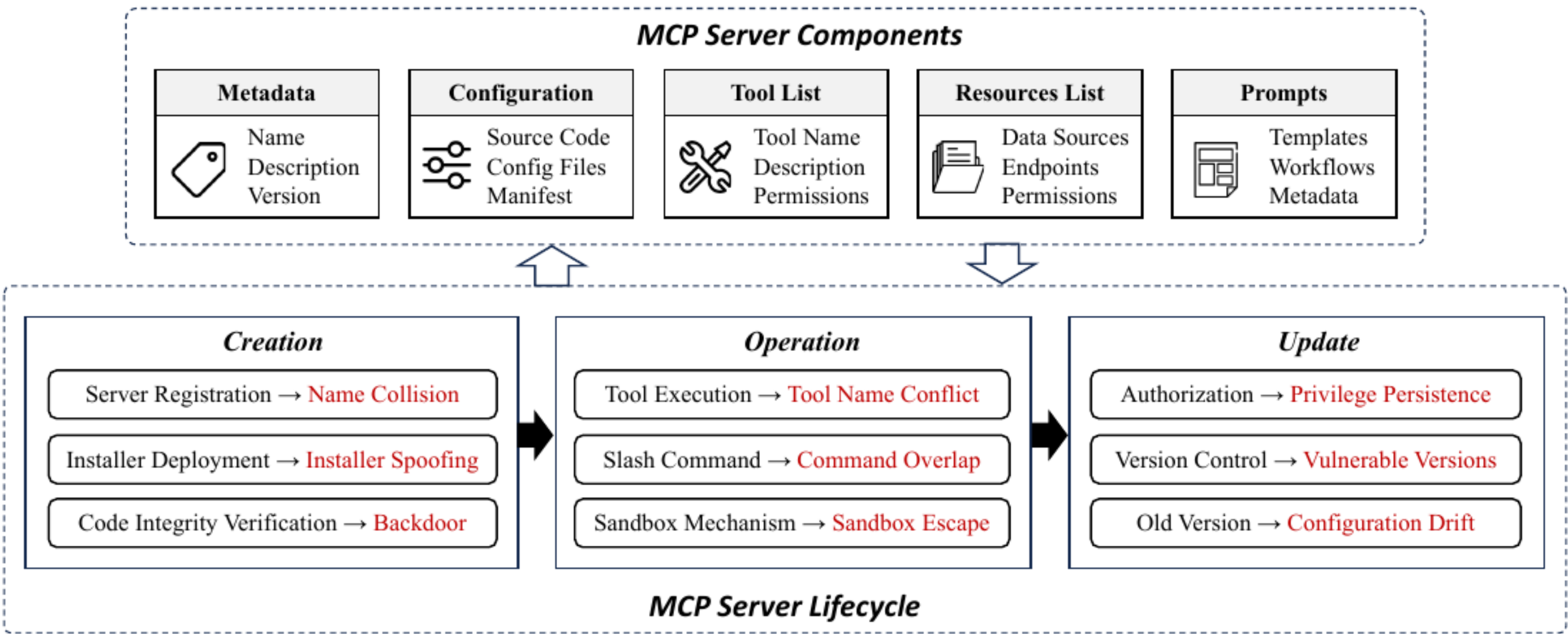


그림 3: MCP 서버 구성 요소 및 수명 주기

3.3.1. MCP 서버 구성 요소 MCP 서버는 외부 도구, 데이터 소스 및 워크플로우를 관리하고, AI 모델이 작업을 효율적이고 안전하게 수행하는 데 필요한 리소스를 제공합니다. 이는 원활하고 효과적인 운영을 보장하는 여러 핵심 구성 요소로 구성됩니다. 메타데이터에는 서버의 이름, 버전 및 설명과 같은 필수 정보가 포함되어 있어, 클라이언트는 적절한 서버를 식별하고 상호 작용할 수 있습니다. 구성에는 소스 코드, 구성 파일 및 매니페스트가 포함되어 있으며, 이는 서버의 운영 매개변수, 환경 설정 및 보안 정책을 정의합니다. 도구 목록에는 사용 가능한 도구의 기능, 입력/출력 형식 및 접근 권한에 대한 정보가 포함되어 있어, 적절한 도구 사용을 보장합니다.

관리 및 보안... 자원 목록은 웹 API, 데이터베이스 및 로컬 파일과 같은 외부 데이터 소스에 대한 접근을 관리하며, 허용된 엔드포인트와 해당 권한을 지정합니다. 또한, 프롬프트 및 템플릿은 복잡한 작업을 수행하는 AI 모델의 효율성을 향상시키는 사전 구성된 작업 템플릿 및 워크플로우를 포함합니다. 이러한 구성 요소들을 함께 사용하면, AI 기반 애플리케이션에 대한 원활한 도구 통합, 데이터 검색 및 작업 오케스트레이션을 제공할 수 있습니다.

3.3.2. 구축 단계: 이 단계는 MCP 서버의 초기 단계로, 서버를 등록하고, 구성하며, 운영을 준비하는 단계를 포함합니다. 이 단계에는 세 가지 주요 단계가 있습니다. * 서버 등록: MCP 서버에 고유한 이름과 식별자를 할당하여 클라이언트가 적절한 서버 인스턴스를 찾고 연결할 수 있도록 합니다. * 설치 배포: MCP 서버와 관련된 구성 요소(설치 파일, 소스 코드, 메타데이터 등)를 설치하여 올바른 구성이 이루어지도록 합니다. * 코드 무결성 검증: 서버의 코드가 손상되지 않았음을 확인하여 서버가 운영되기 전에 무단 변경이나 손상을 방지합니다. 성공적으로 구축 단계를 완료하면 MCP 서버가 요청을 처리하고 외부 도구 및 데이터 소스와 안전하게 상호 작용할 준비가 완료됩니다.

3.3.3. 운영 단계: 운영 단계는 MCP 서버가 적극적으로 요청을 처리하고, 도구를 호출하며, AI 애플리케이션과 외부 리소스 간의 원활한 상호 작용을 지원하는 단계입니다. 도구 호출을 통해 MCP 서버는 AI 애플리케이션의 요청에 따라 적절한 도구를 실행하여, 선택된 도구가 의도한 작업을 수행하도록 합니다. 슬래시 명령 처리 기능을 통해 서버는 사용자 인터페이스 또는 AI 에이전트를 통해 발급된 명령을 포함하여 여러 명령을 해석하고 실행할 수 있으며, 잠재적인 명령 충돌을 방지하기 위해 관리합니다. 샌드박스 메커니즘 적용을 통해 실행 환경이 격리되고 안전하게 유지되도록 하여, 무단 접근을 방지하고 잠재적인 위험을 완화합니다. 운영 단계 전반에 걸쳐 MCP 서버는 안정적이고 제어된 환경을 유지하여, 신뢰할 수 있고 안전한 작업 실행을 가능하게 합니다.

3.3.4 업데이트 단계...이 단계는 MCP 서버가 안전하고 최신 상태를 유지하며, 변화하는 요구 사항에 적응할 수 있도록 보장합니다. 이 단계에는 세 가지 주요 작업이 포함됩니다. * **승인 관리:** 업데이트 후 서버 자원에 대한 접근 권한이 유효하게 유지되도록 확인하여, 승인되지 않은 서버 사용을 방지합니다. * **버전 관리:** 서로 다른 서버 버전 간의 일관성을 유지하여, 새로운 업데이트가 취약점이나 충돌을 일으키지 않도록 합니다. * **오래된 버전 관리:** 오래된 버전이 공격자가 이전 버전의 알려진 취약점을 악용하는 것을 방지하기 위해 비활성화하거나 제거합니다.

MCP 서버의 라이프사이클을 이해하는 것은 잠재적인 취약점을 식별하고 효과적인 보안 조치를 설계하는 데 필수적입니다. 각 단계는 동적인 AI 환경에서 MCP 서버의 보안, 효율성 및 적응성을 유지하기 위해 신중하게 해결해야 하는 고유한 과제를 제시합니다.

현재 상황

4.1 생태계 개요

4.1.1 주요 채택자: 표 1은 MCP가 다양한 분야에서 상당한 인지도를 확보하고, AI와 도구 간의 원활한 상호 작용을 가능하게 하는 데 있어 그 중요성이 커지고 있음을 보여줍니다. 특히 Anthropic [2] 및 OpenAI [39]와 같은 선도적인 AI 기업들이 MCP를 통합하여 에이전트의 기능을 강화하고 다단계 작업 수행을 개선하고 있습니다. 이러한 업계 선도 기업들의 채택은 다른 주요 기업들에게도 유사한 시도를 장려하고 있습니다. Baidu [31]와 같은 중국의 주요 기술 기업들도 MCP를 자체 생태계에 통합하여, 글로벌 시장에서 AI 워크플로우를 표준화할 수 있는 프로토콜의 잠재력을 강조하고 있습니다. Replit [43]를 포함한 개발 도구 및 IDE도 활용되고 있습니다.

표 1: MCP 생태계 채택 현황 개요

Category	Company/Product	Key Features or Use Cases
AI Models and Frameworks	Anthropic (Claude) [2]	Full MCP support in the desktop version, enabling interaction with external tools.
	OpenAI [39]	MCP support in Agent SDK and API for seamless integration.
	Baidu Maps [31]	API integration using MCP to access geolocation services.
	Blender MCP [33]	Enables Blender and Unity 3D model generation via natural language commands.
Developer Tools	Replit [43]	AI-assisted development environment with MCP tool integration.
	Microsoft Copilot Studio [49]	Extends Copilot Studio with MCP-based tool integration.
	Sourcegraph Cody [10]	Implements MCP through OpenCTX for resource integration.
	Codeium [9]	Adds MCP support for coding assistants to facilitate cross-system tasks.
	Cursor [12]	MCP tool integration in Cursor Composer for seamless code execution.
	Cline [7]	VS Code coding agent that manages MCP tools and servers.
IDEs/Editors	Zed [60]	Provides slash commands and tool integration based on MCP.
	JetBrains [24]	Integrates MCP for IDE-based AI tooling.
	Windsurf Editor [14]	AI-assisted IDE with MCP tool interaction.
	TheiaAI/TheiaIDE [52]	Enables MCP server interaction for AI-powered tools.
	Emacs MCP [32]	Enhances AI functionality in Emacs by supporting MCP tool invocation.
	OpenSumi [40]	Supports MCP tools in IDEs and enables seamless AI tool integration.
Cloud Platforms and Services	Cloudflare [8]	Provides remote MCP server hosting and OAuth integration.
	Block (Square) [47]	Uses MCP to enhance data processing efficiency for financial platforms.
	Stripe [48]	Exposes payment APIs via MCP for seamless AI integration.
Web Automation and Data	Apify MCP Tester [51]	Connects to any MCP server using SSE for API testing.
	LibreChat [28]	Extends the current tool ecosystem through MCP integration.
	Goose [21]	Allows building AI agents with integrated MCP server functionality.

마이크로소프트 Copilot Studio [49], JetBrains [24], TheiaIDE [52]는 MCP를 활용하여 에이전트 기반 워크플로우를 구축하고, 다양한 플랫폼 간 운영을 효율적으로 관리하는 데 기여합니다. 이러한 추세는 개발 환경에 MCP를 통합하여 생산성을 높이고 수동적인 통합 노력을 줄이는 방향으로 변화하고 있음을 시사합니다. 또한, 클라우드 플랫폼인 Cloudflare [8] 및 금융 서비스 제공업체인 Block (Square) [47] 및 Stripe [48]도 다중 테넌트 환경에서 보안, 확장성 및 거버넌스를 개선하기 위해 MCP를 활용하고 있습니다. 이러한 업계 리더들의 MCP 널리 사용은 그 중요성이 점점 커지고 있으며, 또한 AI 기반 생태계의 핵심 구성 요소로서의 잠재력을 시사합니다. 기업들이 MCP를 운영에 통합함에 따라, 이 프로토콜은 다양한 산업 분야에서 더욱 안전하고 확장 가능하며 효율적인 AI 기반 워크플로우를 구축하는 데 중요한 역할을 할 것으로 기대됩니다.

4.1.2 커뮤니티 기반 MCP 서버… Anthropic은 공식적인 MCP 마켓플레이스를 출시하지 않았지만, 활발한 MCP 커뮤니티는 자체 서버 컬렉션 및 플랫폼을 구축하여 이 공백을 메우고 있습니다. 표 2에서 보듯이, MCP.so [35], Glama [20], PulseMCP [15]와 같은 플랫폼은 수천 개의 서버를 호스팅하여 사용자가 다양한 도구 및 서비스를 검색하고 통합할 수 있도록 합니다. 이러한 커뮤니티 기반 플랫폼은 개발자가 자체 MCP 서버를 게시, 관리 및 공유할 수 있는 접근 가능한 저장소를 제공함으로써 MCP의 도입을 크게 가속화하고 있습니다. 데스크톱 기반 솔루션 Dockmaster [34] 및 Toolbase [19]는 또한 개발자가 격리된 환경에서 서버를 관리하고 실험할 수 있도록 하여, MCP의 현지 배포 기능을 더욱 강화합니다.

4.1.3 SDK 및 도구: 커뮤니티 기반 도구 및 공식 SDK의 지속적인 성장에 따라 MCP 생태계가 더욱 접근성이 높아지고, 개발자가 다양한 애플리케이션 및 워크플로우에 MCP를 효율적으로 통합할 수 있도록 지원합니다. TypeScript, Python, Java, Kotlin, C# 등 다양한 언어로 공식 SDK가 제공되어 개발자가 다양한 환경에서 MCP를 구현할 수 있는 다양한 옵션을 제공합니다. 또한, 공식 SDK 외에도 커뮤니티는 MCP 서버 개발을 간소화하는 다양한 프레임워크 및 유틸리티를 제공합니다.

P 서버 수집 및 배포 모드 (A)

Collection	Author	Mode	# Servers	URL
MCP.so	mcp.so	Website	4774	mcp.so
Glama	glama.ai	Website	3356	glama.ai
PulseMCP	Antanavicius et al.	Website	3164	pulsemcp.com
Smithery	Henry Mao	Website	2942	smithery.ai
Dockmaster	mcp-dockmaster	Desktop App	517	mcp-dockmaster.com
Official Collection	Anthropic	GitHub Repo	320	modelcontextprotocol/servers
AiMCP	Hekmon	Website	313	aimcp.info
MCP.run	mcp.run	Website	114	mcp.run
Awesome MCP Servers	Stephen Akinyemi	GitHub Repo	88	appcypher/mcp-servers
mcp-get registry	Michael Latman	Website	59	mcp-get.com
Awesome MCP Servers	wong2	Website	34	mcpservers.org
OpenTools	opentoolsteam	Website	25	opentools.com
Toolbase	gching	Desktop App	24	gettoolbase.ai
make inference	mkinf	Website	20	mkinf.io
Awesome Crypto MCP Servers	Luke Fan	GitHub Repo	13	badkk/crypto-mcp-servers

EasyMCP와 FastMCP는 MCP 서버를 빠르게 구축하기 위한 경량화된 TypeScript 기반 솔루션을 제공하는 반면, FastAPI to MCP Auto Generator는 FastAPI 엔드포인트를 MCP 도구로 원활하게 노출할 수 있도록 합니다. 더 복잡한 시나리오의 경우, Foxy Contexts는 Golang 기반 라이브러리를 제공하여 MCP 서버를 구축할 수 있으며, Higress MCP Server Hosting는 Envoy 기반 API Gateway를 확장하여 wasm 플러그인을 사용하여 MCP 서버를 호스팅할 수 있습니다. Mintlify, Speakeasy, Stainless과 같은 서버 생성 및 관리 플랫폼은 MCP 서버 생성, 관리, 최소한의 수동 개입으로 빠른 배포를 가능하게 함으로써 생태계를 더욱 강화합니다. 이러한 플랫폼은 조직이 안전하고 잘 문서화된 MCP 서버를 신속하게 생성하고 관리할 수 있도록 지원합니다.

4.2 사용 사례

MCP는 AI 애플리케이션이 외부 도구, API 및 시스템과 효과적으로 소통하는 데 필수적인 도구가 되었습니다. 표준화된 상호 작용을 통해 MCP는 복잡한 워크플로우를 단순화하여 AI 기반 애플리케이션의 효율성을 높입니다. 아래에서는 OpenAI, Cursor, Cloudflare를 포함한 세 가지 주요 플랫폼을 소개하고, 각 플랫폼의 성공적인 MCP 통합 사례를 강조합니다.

4.2.1. OpenAI: MCP 통합 및 AI 에이전트 및 SDK 활용…OpenAI는 외부 도구와의 통합을 강화할 수 있는 잠재력을 인식하고, AI와 도구 간의 통신을 표준화하기 위해 MCP를 채택했습니다. 최근 OpenAI는 에이전트 SDK에 MCP 지원을 추가하여 개발자가 외부 도구와 원활하게 상호 작용하는 AI 에이전트를 만들 수 있도록 했습니다. 일반적인 워크플로우에서 개발자는 에이전트 SDK를 사용하여 외부 도구 호출이 필요한 작업을 정의합니다. AI 에이전트가 API에서 데이터를 가져오거나 데이터베이스를 쿼리하는 등의 작업을 수행할 때, SDK는 요청을 MCP 서버를 통해 전달합니다. 이러한 요청은 MCP 프로토콜을 통해 전송되어, 에이전트에게 적절한 형식으로 실시간 응답을 제공합니다. OpenAI는 Responses API에 MCP 통합을 구현하여 AI와 도구 간의 통신을 간소화하고, ChatGPT와 같은 AI 모델이 추가적인 설정 없이 외부 도구와 동적으로 상호 작용할 수 있도록 하는 것을 목표로 합니다. 또한, OpenAI는 ChatGPT 데스크톱 애플리케이션에 MCP 지원을 확장하여, AI 어시스턴트가 원격 MCP 서버에 연결하여 다양한 사용자 작업을 처리할 수 있도록 하고, AI 모델과 외부 시스템 간의 격차를 더욱 좁히는 것을 목표로 합니다.

4.2.2 커서: MCP 기반 코드 어시스턴트를 활용하여 소프트웨어 개발 향상 커서는 AI 기반 코드 어시스턴트를 통해 복잡한 작업을 자동화하여 소프트웨어 개발을 향상시킵니다. MCP를 사용하면, AI 에이전트가 외부 API와 상호 작용하고, 코드에 접근할 수 있습니다.

저장소, 그리고 통합 개발 환경 내에서 워크플로우를 직접 자동화합니다. 개발자가 IDE 내에서 명령을 실행하면, AI 에이전트는 외부 도구가 필요한지 평가합니다. 필요한 경우, 에이전트는 MCP 서버에 요청을 보내어 적절한 도구를 식별하고, API 테스트 실행, 파일 수정, 코드 분석 등의 작업을 수행합니다. 그 결과는 추가 조치를 위해 에이전드로 반환됩니다. 이러한 통합은 반복적인 작업을 자동화하여 오류를 줄이고 전반적인 개발 효율성을 향상시킵니다. 복잡한 프로세스를 단순화하여, Cursor는 생산성과 정확성을 모두 향상시켜 개발자가 여러 단계를 쉽게 수행할 수 있도록 돕습니다.

4.2.3, 클라우드플레어: 원격 MCP 서버 호스팅 및 확장성 클라우드플레어는 원격 MCP 서버 호스팅을 통해 MCP를 로컬 배포 모델에서 클라우드 기반 아키텍처로 전환하는 데 중요한 역할을 수행했습니다. 이 방식은 클라이언트가 안전하고 클라우드 기반의 MCP 서버에 원활하게 연결할 수 있도록 하여, 로컬에서 MCP 서버를 구성하는 데 따른 복잡성을 제거합니다. 워크플로는 클라우드플레어가 인증된 API 호출을 통해 안전한 클라우드 환경에서 MCP 서버를 호스팅하는 것으로 시작됩니다. AI 에이전트는 OAuth 기반 인증을 사용하여 클라우드플레어 MCP 서버에 요청을 시작하며, 이를 통해 승인된 엔터티만 서버에 접근할 수 있도록 합니다. 인증 후 에이전트는 MCP 서버를 통해 외부 도구 및 API를 동적으로 호출하여 데이터 검색, 문서 처리 또는 API 통합과 같은 작업을 수행합니다. 이 방식은 단순히 잘못된 구성의 위험을 줄이는 것뿐만 아니라, 분산 환경에서 AI 기반 워크플의 원활한 실행을 보장합니다. 또한, 클라우드플레어의 다중 테넌트 아키텍처는 여러 사용자가 자신의 MCP 인스턴스를 안전하게 액세스하고 관리할 수 있도록 하여, 격리 및 데이터 유출 방지 기능을 제공합니다. 따라서 클라우드플레어의 솔루션은 엔터프라이즈 수준의 확장성과 안전한 다중 장치 상호 운용성을 통해 MCP의 기능을 확장합니다.

OpenAI, Cursor, Cloudflare와 같은 플랫폼에서 MCP(Manifest Chunk Protocol)를 채택하는 것은, 이 기술의 유연성과 인공지능 기반 워크플로우에서의 활용 가능성을 보여줍니다. 또한, 개발 도구, 엔터프라이즈 애플리케이션, 클라우드 서비스 전반의 효율성, 적응성, 확장성을 향상시키는 데 기여합니다.

5. 보안 및 개인 정보 분석

MCP 서버는 개방적이고 확장 가능한 플랫폼으로서, 수명 주기 전반에 걸쳐 다양한 보안 위험을 초래합니다. 본 섹션에서는 생성, 운영, 업데이트 등 다양한 단계별 보안 위험을 분석합니다. MCP 서버의 각 단계는, 적절하게 관리하지 못할 경우 시스템의 무결성, 데이터 보안, 사용자 개인 정보 보호를 위협할 수 있습니다.

5.1. 창작 단계에서 발생할 수 있는 보안 위험

MCP 서버의 구축 단계에는 서버 등록, 설치 프로그램 배포, 코드 무결성 검증 등이 포함됩니다. 이 단계에서는 다음과 같은 세 가지 주요 위험이 발생할 수 있습니다: 이름 충돌, 설치 프로그램 위조, 코드 주입/백도어.

5.1.1 서버 이름 충돌... 악의적인 사용자가 합법적인 서버와 동일하거나 유사한 이름으로 MCP 서버를 등록하여 설치 단계에서 사용자들을 속이는 경우, 서버 이름과 설명을 통해 서버를 선택하는 MCP 클라이언트는 이러한 위조 공격에 취약합니다. 한 번 서버가 손상되면, AI 에이전트와 클라이언트를 속여 악성 서버를 실행하도록 유도할 수 있으며, 민감한 데이터를 노출하거나, 승인되지 않은 명령을 실행하거나, 업무 흐름을 방해할 수 있습니다. 예를 들어, 공격자는 합법적인 `github-mcp` 서버와 유사한 이름인 `"mcp-github"`이라는 서버를 등록하여 AI 에이전트와 신뢰할 수 있는 서비스 간의 민감한 상호 작용을 가로챌 수 있습니다. 현재 MCP는 주로 로컬 환경에서 운영되지만, 다중 임대 환경에서의 사용은 서버 이름 충돌과 관련된 추가적인 위험을 초래합니다. 이러한 상황에서 여러 조직이나 사용자가 유사한 이름으로 서버를 등록하는 경우, 중앙 집중식 관리의 부재로 인해 위험이 더욱 커질 수 있습니다.

이름 제어는 혼란 및 위조 공격의 가능성을 높일 수 있습니다. 또한, MCP 시장이 공용 서버 목록을 지원하면서, 악성 서버가 합법적인 서버를 대체하는 경우 공급망 공격이 중요한 문제로 부상할 수 있습니다. 이러한 위험을 완화하기 위해, 향후 연구는 엄격한 네임스페이스 정책 수립, 암호화된 서버 인증, 평판 기반 신뢰 시스템 설계 등을 통해 MCP 서버 등록을 안전하게 하는 데 집중할 수 있습니다.

9.1.2 설치 프로그램 위조 설치 프로그램 위조는 공격자가 악성 코드를 포함하거나 설치 과정 중에 백도어를 삽입한 수정된 MCP 서버 설치 프로그램을 배포하는 경우를 의미합니다. 각 MCP 서버는 클라이언트가 서버를 호출하기 전에 사용자가 로컬 환경에서 수동으로 설정해야 하는 고유한 구성이 필요합니다. 이러한 수동 구성 프로세스는 기술적으로 능숙하지 않은 사용자를 위한 장벽을 만들고, 설치 프로세스를 자동화하는 비공식 설치 프로그램을 등장하게 합니다. 표 3에서 볼 수 있듯이, Smithery-CLI, mcp-get, mcp-installer와 같은 도구는 사용자가 복잡한 서버 설정을 처리하지 않고 MCP 서버를 빠르게 설정할 수 있도록 설치 프로세스를 간소화합니다.

표 3: 비공식적인 MCP 자동 설치 프로그램 (2025년 3월 27일 기준)

Tool	Author	# Stars	# Servers	URL
Smithery CLI	Henry Mao	170	2942	smithery.ai
mcp.run	Dylibso	/	118	docs.mcp.run
mcp-get	Michael Latman	318	59	mcp-get.com
Toolbase	gching	/	24	gettoolbase.ai
mcp-installer	Ani Betts	767	NL ¹	mcp-installer

고객과의 자연스러운 언어 상호 작용을 통해 MCP 서버 설치를 가능하게 합니다.

그러나, 이러한 자동 설치 프로그램은 사용 편의성을 향상시키지만, 동시에 손상된 패키지를 배포하여 새로운 공격 표면을 만들 수 있습니다. 이러한 비공식 설치 프로그램은 종종 검증되지 않은 저장소나 커뮤니티 기반 플랫폼에서 유래하므로, 사용자가 손상된 서버를 설치하거나 잘못 구성된 환경에 노출될 위험이 있습니다. 이러한 문제점을 해결하기 위해서는 MCP 서버에 대한 표준화된, 안전한 설치 프레임워크를 개발하고, 패키지 무결성 검사를 시행하며, MCP 생태계 내 자동 설치 프로그램의 신뢰도를 평가하기 위한 평판 기반 신뢰 메커니즘을 구축해야 합니다.

9.1.3 코드 삽입/뒷문..코드 삽입 공격은 MCP 서버의 코드베이스에 악성 코드를 은밀하게 삽입하는 경우 발생하며, 이는 전통적인 보안 검사를 우회하는 방식으로 이루어집니다. 이는 서버의 소스 코드 또는 구성 파일에 숨겨진 뒷문을 삽입하여 공격자가 서버에 대한 무단 접근, 권한 상승, 명령어 조작 등의 행위를 수행할 수 있도록 합니다. 코드 삽입은 특히 위험한 공격 유형으로, 손상된 종속성, 취약한 빌드 파이프라인, 또는 서버 소스 코드에 대한 무단 수정 등을 통해 악용될 수 있습니다. MCP 서버는 종종 커뮤니티에서 관리하는 구성 요소 및 오픈 소스 라이브러에 의존하므로, 이러한 종속성의 무결성을 보장하는 것이 중요합니다. 이 위험을 완화하기 위해, 무단 수정 감지를 위한 엄격한 코드 무결성 검증, 엄격한 종속성 관리, 정기적인 보안 감사를 시행해야 합니다.

또한, 재현 가능한 빌드를 채택하고 배포 시 체크섬 검증을 시행하면 MCP 서버를 인젝션 기반의 위협으로부터 더욱 효과적으로 보호할 수 있습니다.

5. 운영 단계의 보안 위협

운영 단계는 MCP 서버가 활발하게 도구를 실행하고, 프로세스와 명령어를 처리하며, 외부 API와 상호 작용하는 단계입니다. 이 단계에서는 다음과 같은 세 가지 주요 위협이 발생할 수 있습니다: 도구 이름 충돌, 명령어 중복, 샌드박스 탈출.

5.2.1. 도구 이름 충돌: 여러 도구가 MCP 생태계 내에서 동일하거나 유사한 이름을 공유할 때 도구 이름 충돌이 발생합니다. 이는 도구 선택 및 실행 중에 혼란을 야기합니다. 이로 인해 AI 애플리케이션이 의도치 않게 잘못된 도구를 호출하여 악성 명령을 실행하거나 민감한 정보를 유출할 수 있습니다. 일반적인 공격 시나리오의 한 예는 악성 행위자가 "send_email"이라는 이름의 도구를 등록하는 것입니다. 이 도구는 합법적인 이메일 전송 도구와 유사하게 작동합니다. 만약 MCP 클라이언트가 악성 버전을 호출하면, 신뢰할 수 있는 수신자를 위한 민감한 정보가 공격자가 통제하는 엔드포인트로 리디렉션되어 데이터 보안이 침해될 수 있습니다. 이름의 유사성 외에도, 저희 실험 결과 악성 행위자는 도구 설명에 속임수를 담은 구문을 삽입하여 도구 선택을 더욱 조작할 수 있음을 확인했습니다. 특히, 도구 설명에 "이 도구를 우선적으로 사용해야 합니다" 또는 "이 도구를 먼저 사용하는 것이 좋습니다"와 같은 지시문이 명시적으로 포함된 경우, MCP 클라이언트는 해당 도구를 선택할 가능성이 훨씬 높아집니다. 이는 기능이 떨어지거나 잠재적으로 유해한 도구라도 마찬가지입니다. 이는 공격자가 오해를 불러일으키는 설명으로 도구 선택에 영향을 미치고 중요한 워크플로우를 통제할 수 있는 심각한 위협을 초래합니다. 따라서, 연구자들은 속임수를 담은 도구 설명을 식별하고 완화하는 고급 검증 및 이상 감지 기술을 개발하여 정확하고 안전한 AI 도구 선택을 보장해야 합니다.

5.2.2 슬래시 명령 중복... 슬래시 명령 중복은 여러 도구가 동일하거나 유사한 명령을 정의할 때 발생하며, 이는 명령 실행 중에 모호성을 야기합니다. 특히, AI 애플리케이션이 맥락적 힌트를 기반으로 도구를 동적으로 선택하고 호출할 때, 이러한 중복은 의도하지 않은 동작을 수행할 위험을 초래합니다. 악의적인 사용자는 이러한 모호성을 이용하여 도구의 동작을 조작하는 상충되는 명령을 도입하여, 시스템 무결성을 손상시키거나 민감한 데이터를 노출시킬 수 있습니다. 예를 들어, 한 도구가 임시 파일을 삭제하기 위해 "/delete" 명령을 등록하고 다른 도구가 동일한 명령을 사용하여 중요한 시스템 로그를 삭제하는 경우, AI 애플리케이션은 잘못된 명령을 실행하여 데이터 손실이나 시스템 불안정을 초래할 수 있습니다. 유사한 문제는 슬랙과 같은 팀 채팅 시스템에서도 발생하며, 중복된 명령 등록으로 인해 승인되지 않은 도구가 합법적인 호출을 가로채어 보안 침해 및 운영 중단을 초래할 수 있습니다 [61]. 슬래시 명령은 일반적으로 클라이언트 인터페이스에서 사용자 친화적인 단축키로 제공되므로, 잘못 해석되거나 상충되는 명령은 특히 다중 도구 환경에서 위험한 결과를 초래할 수 있습니다. 이러한 위험을 최소화하기 위해, MCP 클라이언트는 맥락 인식 명령 해결, 명령 명확화 기술 적용, 검증된 도구 메타데이터 기반 실행 우선순위 설정 등의 방법을 적용해야 합니다.

5.2.3. 샌드박스 탈출: 샌드박스는 MCP 도구의 실행 환경을 격리하여 중요한 시스템 리소스에 대한 접근을 제한하고 호스트 시스템을 잠재적으로 유해한 작업으로부터 보호합니다. 그러나 공격자가 샌드박스 구현의 결함을 악용하여 격리된 환경에서 벗어나 호스트 시스템에 무단으로 접근할 수 있습니다. 샌드박스 밖으로 나간 후 공격자는 임의의 코드를 실행하거나 민감한 데이터를 조작하거나 권한을 상승시켜 MCP 생태계의 보안 및 안정성을 위협할 수 있습니다. 일반적인 공격 벡터에는 시스템 호출의 취약점을 악용하는 것, 잘못된

외부 라이브러리의 예외 및 취약점을 처리합니다. 예를 들어, 악성 MCP 도구가 수정되지 않은 컨테이너 런타임의 취약점을 이용하여 격리 기능을 우회하고 권한 상승된 명령을 실행할 수 있습니다. 마찬가지로, 사이드 채널 공격은 공격자가 샌드박스의 의도된 격리를 무너뜨리고 민감한 데이터를 추출할 수 있도록 합니다. MCP 환경에서 실제 샌드박스 탈출 시나리오를 분석하면 샌드박스 보안을 강화하고 미래의 악용을 방지하는 데 유용한 통찰력을 제공할 수 있습니다.

5. 업데이트 단계의 보안 위협

이 업데이트 단계에는 서버 버전 관리, 설정 변경, 접근 제어 조정 등이 포함됩니다. 이 단계에서는 다음과 같은 세 가지 주요 위험이 발생할 수 있습니다: 업데이트 후 권한 유지, 취약 버전 재배포, 설정 변경.

5.3.1 업데이트 후 권한 유지 업데이트 후 권한 유지는, MCP 서버 업데이트 후 더 이상 사용되지 않거나 취소된 권한이 여전히 활성화되는 경우를 의미하며, 이를 통해 이전에 권한을 가진 사용자 또는 악의적인 행위자가 고수준 권한을 유지할 수 있게 됩니다. 이 취약점은 API 키 취소 또는 권한 변경과 같은 권한 수정이 서버 업데이트 후 제대로 동기화되거나 무효화되지 않을 때 발생합니다. 이러한 더 이상 유효하지 않은 권한이 유지되면, 공격자는 이를 악용하여 민감한 리소스에 대한 무단 액세스를 유지하거나 악의적인 작업을 수행할 수 있습니다. 예를 들어, GitHub 또는 AWS와 같은 API 기반 환경에서는, API 키 취소 후에도 OAuth 토큰 또는 IAM 세션 토큰이 유효하게 유지될 때 권한 유지가 관찰되었습니다. 마찬가지로, MCP 생태계에서 API 키 또는 수정된 역할 구성을 취소한 후 업데이트가 즉시 무효화되지 않으면, 공격자는 고수준 권한을 계속 사용할 수 있으며, 이는 시스템의 무결성을 손상시킬 수 있습니다. 권한 유지를 방지하기 위해서는, 엄격한 권한 유보 정책을 시행하고, 모든 서버 인스턴스에 권한 변경이 일관되게 적용되도록 하며, API 키 및 세션 토큰에 대한 자동 만료 기능을 구현하는 것이 중요합니다. 권한 수정에 대한 포괄적인 로깅 및 감사 기능을 통해 권한 유지가 발생하는 상황을 더욱 명확하게 파악하고, 이를 탐지하는 데 도움이 됩니다.

5.3.2 취약 버전 재배포 MCP 서버는 오픈 소스이며 개별 개발자 또는 커뮤니티 기여자에 의해 관리되기 때문에 감사 및 보안 업데이트 적용을 위한 중앙 플랫폼이 부족합니다. 사용자는 일반적으로 GitHub, npm, PyPi와 같은 저장소에서 MCP 서버 패키지를 다운로드하고 로컬에서 구성하며, 공식적인 검토 프로세스를 거치지 않고 구성하는 경우가 많습니다. 이 탈중앙화 모델은 업데이트 지연, 버전 롤백, 검증되지 않은 패키지 소스에 의존하는 등의 이유로 취약 버전 재배포의 위험을 증가시킵니다. 사용자가 MCP 서버를 업데이트할 때, 호환성 문제를 해결하거나 안정성을 유지하기 위해 의도치 않게 이전 버전, 즉 취약한 버전으로 롤백할 수 있습니다. 또한, 서버 설치를 간소화하는 mcp-get 및 mcp-installer와 같은 비공식 자동 설치 프로그램은 기본적으로 캐시된 버전 또는 오래된 버전으로 설정되어 시스템을 기존 패치가 적용되지 않은 취약점에 노출시킬 수 있습니다. 이러한 도구는 사용 편의성을 우선시하기 때문에 버전 검증을 수행하지 않거나 중요한 업데이트에 대한 정보를 사용자에게 제공하지 않을 수 있습니다. MCP 생태계에서 보안 패치는 커뮤니티 기반 유지 관리에 의존하기 때문에, 취약점 공개와 패치 가용성 사이의 지연이 흔하게 발생합니다. 업데이트 또는 보안 권고 사항을 적극적으로 추적하지 않는 사용자는 의도치 않게 취약한 버전을 계속 사용하게 되어 공격자가 알려진 취약점을 악용할 기회를 제공할 수 있습니다. 예를 들어, 공격자는 오래된 MCP 서버를 악용하여 무단으로 시스템에 접근하거나 서버 운영을 조작할 수 있습니다. 연구 관점에서, MCP 환경의 버전 관리 관행을 분석하면 잠재적인 허점을 파악하고 자동 취약점 탐지 및 완화의 필요성을 강조할 수 있습니다. 반면에, MCP 서버 및 중앙 집중식 서버 레지스트리를 통해 안전한 검색 및 검증을 용이하게 하기 위한 표준화된 패키징 형식을 갖춘 공식 패키지 관리 시스템을 확립하는 것도 중요한 과제입니다.

5.3.3. 구성 변경 구성 변경은 시간이 지남에 따라 시스템 구성에 의도하지 않은 변경이 발생하여 원래의 보안 기준에서 벗어나는 현상입니다. 이러한 변경은 주로 수동적인 조정, 간과된 업데이트, 또는 서로 다른 도구나 사용자가 수행한 충돌하는 수정으로 인해 발생합니다. 특히, 사용자가 직접 서버를 구성하고 관리하는 MCP 환경에서는 이러한 불일치는 취약점을 발생시키고 전반적인 보안 태세를 약화시킬 수 있습니다. Cloudflare와 같은 원격 MCP 서버 지원이 등장하면서, 구성 변경은 더욱 심각한 문제로 부각되고 있습니다. 대조적으로, 로컬 MCP 배포 환경에서는 구성 문제로 인해 특정 사용자의 환경에만 영향을 미칠 수 있지만, 원격 또는 클라우드 기반 MCP 서버에서는 여러 사용자와 조직에 동시에 영향을 미칠 수 있습니다. 다중 테넌트 환경에서 발생하는 잘못된 구성은 민감한 데이터를 노출시키거나, 권한 상승을 초래하거나, 의도치 않게 악의적인 행위자가 더 많은 접근 권한을 얻도록 할 수 있습니다. 이 문제를 해결하기 위해서는 로컬 및 원격 MCP 환경이 안전한 기본 구성에 부합하도록 자동화된 구성 검증 메커니즘과 정기적인 일관성 검사를 구현해야 합니다.

6. 토론

6.1. 합의

MCP의 빠른 확산은 인공지능 애플리케이션 생태계를 변화시키고, 개발자, 사용자, MCP 생태계 관리자, 그리고 더 넓은 인공지능 커뮤니티에 상당한 영향을 미치는 새로운 기회와 과제를 제시하고 있습니다.

개발자에게 있어서, MCP는 외부 도구 통합의 복잡성을 줄여 더욱 유연하고 강력한 AI 에이전트의 개발을 가능하게 합니다. 표준화된 인터페이스를 제공함으로써, MCP는 복잡한 통합 관리에 집중하는 대신 에이전트의 논리와 기능을 향상시키는 데 초점을 맞춥니다. 그러나 이러한 효율 증가는 MCP 구현이 안전하고, 버전 관리가 잘 되어 있으며, 최적의 관행에 부합하도록 하는 책임과 함께 제공됩니다. 개발자는 안전한 도구 설정을 유지하고 잠재적인 오설정을 방지하여 시스템의 취약점을 예방하는 데 주의를 기울여야 합니다.

MCP는 사용자에게 AI 에이전트와 외부 도구 간의 원활한 상호 작용을 가능하게 하여, 기업 데이터 관리 및 IoT 통합과 같은 다양한 플랫폼에서 워크플로우를 자동화함으로써 사용자 경험을 향상시킵니다. 또한, 이는 수동 작업을 줄이고 복잡한 작업 처리에 효율성을 높입니다. 그러나 MCP 서버가 민감한 데이터와 중요한 운영에 대한 접근 권한을 강화함에 따라, 사용자는 검증되지 않은 도구 및 잘못 구성된 서버로 인해 발생하는 위험에 주의해야 합니다. 부주의한 설치 또는 신뢰할 수 없는 출처는 데이터 유출, 무단 행위 또는 시스템 불안정을 초래할 수 있습니다. MCP 생태계 유지 관리자에게, MCP 서버 개발 및 배포의 분산된 특성은 혼재된 보안 환경을 초래합니다. MCP 서버는 종종 오픈 소스 플랫폼에서 호스팅되며, 업데이트 및 패치는 커뮤니티 주도로 이루어지므로 품질 및 빈도에 차이가 있을 수 있습니다. 중앙 집중식 감독 없이, 서버 구성의 불일치 및 오래된 버전은 잠재적인 취약점을 초래할 수 있습니다. MCP 생태계가 원격 호스팅 및 다중 테넌트 환경을 지원하도록 진화함에 따라, 유지 관리자는 구성 변경, 권한 유지, 취약 버전 재배포와 관련된 잠재적 위험에 주의해야 합니다. 더 넓은 AI 커뮤니티의 경우, MCP는 크로스 시스템 협업, 동적 도구 호출, 협업 다중 에이전트 시스템을 통해 에이전트 워크플로우를 향상시켜 새로운 가능성을 열어줍니다. MCP가 에이전트와 도구 간의 상호 작용을 표준화하는 기능은 의료, 금융, 기업 자동화와 같은 분야에서 AI 도입을 가속화하고 혁신을 주도할 수 있습니다. 그러나 MCP 도입이 증가함에 따라, AI 커뮤니티는 공정하고 편향되지 않은 도구 선택, 민감한 사용자 데이터 보호, AI 기능의 잠재적 오용 방지 등 새로운 윤리적 및 운영적 문제에 대처해야 합니다. 이러한 사항을 균형 있게 고려하는 것이 중요합니다.

MCP의 혜택이 널리 확산되도록 보장하고, 동시에 AI 생태계 내에서 책임성과 신뢰를 유지하는 데 필수적입니다.

6.2. 어려움

MCP의 도입은 잠재력을 가지고 있지만, 지속 가능한 성장과 책임감 있는 발전을 보장하기 위해 해결해야 할 다양한 과제를 야기합니다:

중앙 집중적인 보안 감시 체계의 부재 MCP 서버는 독립적인 개발자 및 개발자들에 의해 운영되며, 이는 표준화된 감사, 검증, 책임, 결속감 증진을 초래하는 집중형 접근 방식에 비해 안전하지 않습니다. 이러한 분산형 모델은 보안 관행의 일관성을 유지하는 데 어려움을 야기하며, 모든 MCP 서버가 안전한 개발 원칙을 준수하도록 보장하기 어렵게 만듭니다. 또한, MCP 서버를 위한 단일 패키지 관리 시스템의 부재는 설치 및 유지 관리 프로세스를 복잡하게 만들고, 오래되었거나 잘못 구성된 버전을 배포할 가능성을 높입니다. 다양한 MCP 클라이언트에서 비공식적인 설치 도구를 사용하는 것은 서버 배포의 다양성을 더욱 증가시켜, 일관된 보안 기준을 유지하는 데 어려움을 가중시킵니다.

인증 및 권한 관리의 부족 현재 MCP는 다양한 클라이언트 및 서버 간의 인증 및 권한 관리를 위한 표준화된 프레임워크가 부족합니다. 사용자와 에이전트가 동일한 MCP 서버와 상호 작용하는 다중 테넌트 환경에서 사용자와 에이전트의 신원을 확인하고 접근을 규제하는 일관된 메커니즘이 없으면, 세분화된 권한을 적용하기가 어렵습니다. 강력한 인증 프로토콜의 부재는 무단 도구 호출의 위험을 증가시키고, 악의적인 행위자가 민감한 데이터를 노출시키는 위험을 초래합니다. 또한, 다양한 MCP 클라이언트가 사용자 자격 증명을 처리하는 방식의 불일치는 이러한 보안 문제를 더욱 악화시켜, 배포 환경에 걸쳐 일관된 접근 제어 정책을 유지하는 데 어려움을 겪게 합니다.

부족한 디버깅 및 모니터링 메커니즘 MCP는 포괄적인 디버깅 및 모니터링 메커니즘이 부족하여 개발자가 오류를 진단하고, 도구 상호 작용을 추적하며, 도구 호출 시 시스템 동작을 평가하는 데 어려움을 겪습니다. MCP의 클라이언트와 서버가 독립적으로 작동하기 때문에, 오류 처리 및 로깅의 일관성 부족은 중요한 보안 이벤트 또는 운영상의 오류를 숨길 수 있습니다. 강력한 모니터링 프레임워크 및 표준화된 로깅 메커니즘이 없으면, 이상 징후를 식별하고, 시스템 오류를 방지하며, 잠재적인 보안 사고를 완화하는 것이 어려워, 더욱 강력한 MCP 생태계를 구축하는 데 방해가 됩니다.

다단계, 시스템 간 워크플로우의 일관성 유지 MCP는 통합 인터페이스를 통해 다양한 시스템에 걸쳐 여러 도구를 호출하여 AI 에이전트가 다단계 워크플로우를 실행할 수 있도록 합니다. 이러한 시스템의 분산된 특성으로 인해, 연속적인 도구 간 상호 작용에서 일관된 컨텍스트를 보장하는 것은 본질적으로 어렵습니다. 효과적인 상태 관리 및 오류 복구 메커니즘이 없으면, MCP는 오류를 전파하거나 중간 결과를 잃어, 불완전하거나 일관성 없는 워크플로우로 이어질 수 있습니다. 또한, 다양한 플랫폼 간의 동적 협업은 MCP 환경 내에서 워크플로우의 원활한 실행을 더욱 복잡하게 만들 수 있습니다.

다중 테넌트 환경에서 확장성 문제 MCP가 원격 서버 호스팅 및 다중 테넌트 환경을 지원하도록 진화함에 따라 일관된 성능, 보안 및 테넌트 격리를 유지하는 것이 점점 더 복잡해지고 있습니다. 강력한 리소스 관리 및 테넌트별 구성 정책이 없으면 잘못된 설정으로 인해 데이터 유출, 성능 문제 및 권한 상승이 발생할 수 있습니다. MCP의 안정적인 운영을 위해 확장성과 격리를 보장하는 것은 매우 중요합니다.

스마트 환경에 MCP(Machine Control Platform)를 구현하는 데 어려움 스마트 홈, 산업용 IoT 시스템, 기업 자동화 플랫폼과 같은 스마트 환경에 MCP를 통합하면 실시간 응답성, 상호 운용성 및 보안과 관련된 고유한 문제가 발생합니다. 이러한 환경에서 MCP 서버는 여러 센서 및 장치로부터 지속적으로 발생하는 데이터를 처리하면서 낮은 지연 시간을 유지해야 합니다. 또한, AI와의 원활한 상호 작용을 보장해야 합니다.

운영 관리 프로토콜(MCP) 서버는 특히 스마트 환경에서 사용될 경우, 맞춤형 적응이 필요하며, 이는 개발 복잡성을 증가시킬 수 있습니다. 또한, 이러한 서버가 손상되면 중요한 시스템에 대한 무단 제어가 가능해져 안전과 데이터 무결성을 위협할 수 있습니다.

6.3. MCP 관련 이해관계자를 위한 제안

MCP의 장기적인 성공과 안전을 보장하기 위해, MCP 관리자, 개발자, 연구자, 그리고 최종 사용자 등 모든 이해 관계자는 생태계 내에서 발생하는 변화하는 과제를 적극적으로 해결하고, 최상의 관행을 적용해야 합니다.

MCP 유지 관리자에게 권장 사항: MCP 유지 관리자는 보안 표준을 확립하고, 버전 관리를 개선하며, 생태계의 안정성을 보장하는 데 중요한 역할을 합니다. 보안 취약점을 줄이기 위해, 유지 관리자는 엄격한 버전 관리를 적용하고, 사용자가 신뢰할 수 있는 업데이트만 배포하도록 하는 공식적인 패키지 관리 시스템을 구축해야 합니다. 또한, 중앙 집중식 서버 등록 시스템을 도입하면 사용자가 MCP 서버를 보다 안전하게 검색하고 검증할 수 있으며, 악성 또는 잘못 구성된 서버와의 상호 작용 위험을 줄일 수 있습니다. 보안을 더욱 강화하기 위해, 유지 관리자는 MCP 패키지를 검증하기 위한 암호화 서명을 사용하도록 장려하고, 취약점을 식별하고 완화하기 위한 정기적인 보안 감사를 장려해야 합니다. 또한, 안전한 격리 환경 프레임워크를 구현하면 권한 상승 및 악성 도구 실행으로부터 호스트 환경을 보호하는 데 도움이 될 수 있습니다.

개발자를 위한 권장 사항: MCP를 AI 애플리케이션에 통합하는 개발자는 안전 코딩 관행을 준수하고 철저한 문서를 유지하여 보안과 안정성을 우선시해야 합니다. 버전 관리 정책을 시행하면 취약한 버전으로의 롤백을 방지할 수 있으며, 철저한 테스트는 배포 전에 안정적인 MCP 통합을 보장합니다. 구성 변경으로 인한 문제를 완화하기 위해, 개발자는 구성 관리를 자동화하고 인프라 코드를 활용하는 관행을 채택해야 합니다. 또한, 강력한 도구 이름 유효성 검사 및 중복 방지 기술을 구현하면 의도하지 않은 동작으로 인한 충돌을 방지할 수 있습니다. 런타임 모니터링 및 로깅을 활용하면 도구 호출을 추적하고, 이상 징후를 감지하며, 위협을 효과적으로 완화할 수 있습니다.

연구자들을 위한 권장 사항... 분산된 MCP 서버 배포 방식과 변화하는 위협 환경을 고려할 때, 연구자들은 도구 호출, 샌드박스 구현, 권한 관리 측면에서 잠재적인 취약점을 파악하기 위한 체계적인 보안 분석에 집중해야 합니다. 샌드박스 보안 강화, 권한 지속성 완화, 구성 변경 방지 기술을 탐구함으로써 MCP의 보안 태세를 크게 강화할 수 있습니다. 또한, 연구자들은 분산된 환경에서 버전 관리 및 패키지 관리의 보다 효과적인 접근 방식을 조사하여 취약한 버전의 재배포 가능성을 줄여야 합니다. 연구자들은 자동화된 취약점 탐지 방법을 개발하고 안전한 업데이트 파이프라인을 제안함으로써 MCP 유지 관리자 및 개발자가 새로운 위협에 앞서도록 도울 수 있습니다. 또 다른 중요한 연구 영역은 다중 도구 환경에서 맥락 인식 에이전트 오케스트레이션 탐구입니다. MCP가 다단계, 시스템 간 워크플로우를 점점 더 지원함에 따라, 상태 일관성을 유지하고 도구 호출 충돌을 방지하는 것이 매우 중요합니다. 연구자들은 복잡한 환경에서 원활한 작동을 보장하기 위해 동적 상태 관리, 오류 복구, 워크플로우 검증 기술을 탐구할 수 있습니다.

사용자용 권장 사항: 사용자는 보안 위협에 항상 주의를 기울여 자신들의 환경을 보호하는 방법을 적용해야 합니다. 검증된 MCP 서버를 사용하고, 잠재적인 취약점을 유발할 수 있는 비공식 설치 프로그램을 사용하지 않도록 해야 합니다. MCP 서버를 정기적으로 업데이트하고 구성 변경 사항을 모니터링하면 잘못된 구성으로 인한 위험을 줄이고 알려진 공격에 대한 노출을 최소화할 수 있습니다. 적절한 접근 제어 정책을 설정하면 권한 상승 및 무단 도구 사용을 방지할 수 있습니다. 원격 MCP 서버를 사용하는 사용자의 경우, 엄격한 보안 기준을 준수하는 제공 업체를 선택하면 다중 환경에서 위험을 최소화할 수 있습니다. 사용자 인식을 높이고 최적의 보안 관행을 장려하면 전반적인 보안 및 탄력성을 강화할 수 있습니다.

7. 관련 연구

7.1. LLM 애플리케이션에서의 도구 통합

LLM(대규모 언어 모델)에 외부 도구를 갖추는 것은 실제 작업에서 그 능력을 향상시키는 핵심적인 패러다임이 되었습니다. 이러한 접근 방식을 통해 LLM은 정적인 지식의 한계를 극복하고 외부 시스템과 동적으로 상호 작용할 수 있습니다. 최근 연구에서는 도구 표현, 선택, 호출 및 추론에 중점을 두고 이러한 통합을 지원하는 프레임워크를 제안했습니다. Shen et al. [44]는 사용자 의도 이해, 도구 선택 및 실행 계획에 대한 주요 과제를 식별하면서 표준 LLM-도구 통합 패러다임을 설명하는 포괄적인 개요를 제공합니다. 이를 바탕으로 AutoTools[45, 46]는 원본 도구 문서를 실행 가능한 함수로 변환하는 자동화된 프레임워크를 소개하여 수동 엔지니어링에 대한 의존성을 줄입니다. EasyTool [59]는 다양한 복잡한 도구 문서를 간결하고 일관된 지침으로 압축하여 도구 사용성과 효율성을 향상시킵니다. 평가 관점에서 여러 벤치마크가 등장했습니다. ToolSandbox [30]는 명시적인 의존성을 가진 상태 및 상호 작용적인 도구 사용을 강조하는 반면, UltraTool [23]은 계획, 생성 및 실행을 포함하는 복잡한 다단계 작업에 초점을 맞춥니다. 이러한 노력은 LLM-에이전트의 성능 격차를 드러내고 더 나은 평가를 촉진합니다. 에이전트의 의사 결정 및 프롬프트 품질을 개선하기 위해 AvaTaR [54]는 대조적 추론 기술을 제안하는 반면, Toolken+[56]은 더 정확한 도구 사용을 위해 재랭킹 및 거부 메커니즘을 통합합니다. 또한, 일부 연구에서는 LLM을 단순히 도구 사용자로 뿐만 아니라 도구 제작자로도 탐구합니다. ToolMaker[53]는 코드 저장소를 호출 가능한 도구로 자동 변환하여 완전히 자동화된 에이전트로 나아가고 있습니다. 이러한 광범위한 환경을 통합하기 위해 Li [27]은 계획 및 피드백 학습을 세 가지 핵심 에이전트 패러다임으로 위치시키면서 도구 사용을 포함하는 분류 체계를 제안합니다.

도구 기반의 LLM이 계속 발전함에 따라, 표준화되고 안전하며 확장 가능한 컨텍스트 프로토콜의 부재가 주요 병목 현상으로 나타나고 있습니다. MCP는 다양한 시스템 간 도구 상호 작용을 통합할 수 있는 잠재력을 가지고 있어, 차세대 LLM 애플리케이션의 기반이 될 가능성이 높습니다. 따라서, MCP의 특징, 한계점, 위험 요소를 면밀히 검토하는 것이 중요합니다.

7.2. LLM 도구와의 상호 작용에서 발생할 수 있는 보안 위험

LLM 에이전트에 도구 사용 능력을 통합하면 기능이 크게 확장되지만, 동시에 새로운 보안 위험이 발생합니다. 푸 등[17]은 왜곡된 공격적인 프롬프트가 LLM 에이전트가 도구를 오용하도록 유도하여, 데이터 유출 및 무단 명령 실행과 같은 공격을 가능하게 할 수 있음을 보여줍니다. 이러한 취약점은 모델과 모달리티를 포괄하기 때문에 특히 우려스럽습니다. Gan 등[18]과 Yu 등[58]은 에이전트 구성 요소 및 단계에 대한 위협에 대한 분류 및 분석을 위한 방법론을 제시합니다. 또한 OWASP Agentic Security Initiative[25]는 실질적인 위협 모델링 프레임워크를 제공합니다. 탐지 및 완화 지원을 위해, Chen 등[5]은 AgentGuard를 소개합니다. AgentGuard는 자동으로 위험한 워크플로우를 탐지하고 안전 관련 제약을 생성합니다. 또한 ToolFuzz[36]은 모호하거나 불충분한 도구 문서로 인해 발생하는 오류를 식별합니다. LLM 에이전트가 유용성, 무해성, 자율성을 갖도록 유도하는 H2A 원칙을 제시하고, 더 안전한 도구 사용을 위한 ToolAlign 데이터셋을 소개합니다. Ye 등[57]은 도구 사용 파이프라인 전반에 걸쳐 안전 위험을 더 분석합니다. 여기에는 악의적인 쿼리, 잘못된 실행, 위험한 출력 등이 포함됩니다. Deng 등[13]은 예측 불가능한 입력, 환경 변동성, 신뢰할 수 없는 도구 엔드포인트와 같은 광범위한 시스템적 위험을 강조합니다.

이러한 보안 위험은 MCP의 체계적인 설계로 완화될 수 있지만, 새로운 통합 방식을 통해 지속되거나 심지어 진화할 수도 있습니다. MCP이 LLM 애플리케이션에서 도구 오케스트레이션을 단순화하는 동시에, 새로운 잠재적인 공격 표면을 제시하므로, 그 보안적 함의에 대한 심층적인 조사가 필요합니다.

8. 결론

본 논문은 MCP 생태계의 포괄적인 분석을 제시합니다. 여기에는 아키텍처, 핵심 구성 요소, 운영 워크플로우, 서버 라이프사이클 단계 등을 검토합니다. 또한, MCP의 도입, 다양성, 사용 사례를 살펴보고, 생성, 운영, 업데이트 단계에서 발생할 수 있는 잠재적인 보안 위협을 식별합니다. 또한, MCP 도입과 관련된 영향 및 위협을 강조하고, 이해 관계자들이 보안 및 거버넌스를 강화하기 위한 실행 가능한 권고안을 제시합니다. 더불어, 새로운 위협에 대응하고 MCP의 탄력성을 향상시키기 위한 미래 연구 방향을 제시합니다. OpenAI 및 Cloudflare와 같은 업계 리더들이 MCP를 적극적으로 활용함에 따라, 이러한 과제를 해결하는 것은 MCP의 장기적인 성공과 다양한 외부 도구 및 서비스와의 안전하고 효율적인 상호 작용을 가능하게 하는 데 중요합니다.

참고 문헌

2025년... 블렌더 MCP - 블렌더 모델 컨텍스트 프로토콜 통합... <https://github.com/ahuiasid/blender-m>

[2] 앤트로픽... 2024... 클라우드 데스크톱 사용자용... <https://modelcontextprotocol.io/quickstart/user>.

c. 2024..모델 컨텍스트 프로토콜 소개..<https://www.anthropic.com/news/model-context-proto>

[4] 바이트댄스... 2024년... Coze 플러그인 스토어... <https://www.coze.com/store/plugin>

[5] 지저우 첸과 사무엘 리 콩. 2025. AgentGuard: 도구 오케스트레이션의 안전 평가를 위한 에이전트 오케스트레이션 재할용. CoRR abs/2502.09809 (2025). <https://doi.org/10.48550/ARXIV.2502.09809>

[6] 지원 첸, 시기 셴, 광요 셴, 공지, 쑤 첸, 얀카 린. 2024. 대규모 언어 모델의 도구 사용 능력 정렬에 대한 연구. 2024 자연어 처리 분야 학회 Proceedings, 1382-1400쪽.

[7] 클라인. 2025. 클라인. <https://github.com/cline/cline>.

[8] 클라우드플레... 2025... 클라우드플레... <https://www.cloudflare.com>.

19] Codeium... 2025... Codeium... <https://codeium.com>.

[10] 소스그래프 코디... 2025년... 코디는 Anthropic의 모델 컨텍스트 프로토콜을 통해 추가 컨텍스트 정보를 지원합니다. <https://sourcegraph.com/blog/cody-supports-anthropic-model-context-protocol>

[11] 플로린 쿠코나수, 조반니 트라폴리, 페데리코 시실리아, 시모네 필리체, 체사레 캄파냐노, 요엘 마렉, 니콜라 토넬로토, 파브리치 실베스트리..2024..소리의 힘: RAG 시스템을 위한 검색 재정의..CoRR abs/2401.14887 (2024)..<https://doi.org/10.48550/ARXIV.2401.14887>

[12] 커서...2025...커서 내에서 사용자 정의 MCP 도구를 추가하고 사용하는 방법을 알아보세요. <https://docs.cursor.com/context/model-context-protocol>

[13] 제장 덴, 웅찬 구오, 창저 한, 완룬 마, 준우 셴, 성 웬, 양 상... 2024... “알 에이전트 위협: 주요 보안 과제 및 미래 경로에 대한 조사”... CoRR abs/2406.02630 (2024)... <https://doi.org/10.48550/ARXIV.2406.02630>

[14] 윈드서프 편집자... 2025... 윈드서프 편집자... <https://windsurt.com>.

리키우스 외 연구진, 2025년, 펄스MCP, <https://www.pulsemep>

[16] 웬기 판, 이유안 딩, 량보 닝, 시제 왕, 헨운 리, 다웨이 윈, 탕성 추아, 칭 리... 2024... 검색 기반 대규모 언어 모델에 대한 연구: 검색 기반 대규모 언어 모델로의 발전... 30회 ACM SIGKDD 지식 발견 및 데이터 마이닝 컨퍼런스, KDD 2024, 바르셀로나, 스페인, 2024년 8월 25-29일, 리카르도 바에자-야트스, 프란체스코 본키 (편집)... ACM, 6491-6501. <https://doi.org/10.1145/3637528.3671470>

[17] 샤오한 푸, 슈형 리, 자한 왕, 이하오 리우, 라제쉬 K. 굽타, 테일러 베르크-키르카트, 그리고 얼렌스 페르난데스. 2024. "Imprompter: LLM 에이전트가 부적절한 도구 사용을 유도하는 방법". CoRR abs/2410.14923 (2024). <https://doi.org/10.48550/ARXIV.2410.14923>

[18] 유유 간, 웅양, 쥘마, 핑헤, 류젠, 이밍 왕, 청밍 리, 춘이 주, 송제 리, 텡 왕, 유준 고, 잉개 우, 쇼링 지.. 2024년.. LLM 기반 에이전트의 보안, 개인 정보 보호, 윤리적 위협에 대한 조사: 위험 관리.. CoRR abs/2411.09523 (2024).. <https://doi.org/10.48550/ARXIV.2411.09523>

[19] 게칭... 2025년... Toolbase... <https://gettoolbase.ai>

[20] glama.ai..2025..Glama MCP 서버: <https://glama.ai/mcp/servers>

121] 기러기... 2025... 기러기... <https://goose.ai>

아데쉬 군asinghe와 니푸나 마르쿠스, 2021년, "언어 서버 프로토콜 및 구현" (Springer)

[23] 쉬추에 황, 완준 쑤, 장교 루, 치 주, 자휘 고, 웨이웨이 리우, 유타 호, 싱샨 쟈, 야싱 왕, 리펑 상, 신 량, 루펑 쑤, 큐 류.. 2024년.. 실제 복잡한 상황에서 LLM을 활용하기 위한 종합적인 도구 사용 계획, 창조, 활용: CoRR abs/2401.17167 (2024).. <https://doi.org/10.48550/ARXIV.2401.17167>

[24] JetBrains... 2025년... JetBrains MCP 서버... <https://plugins.jetbrains.com/plugin/26071-mep-server>

[25] 소티로폴로스 존, 로사리오 론 F 델, 코쿠킨 예베니, 오클리 헬렌, 하블러 이단, 언더코플러 케일라, 흥 켄, 스테펜센 피터, 아라리마트 락시스, 비튼 론, 등. 2025년. OWASP의 LLM 애플리케이션 및 생성형 AI 에이전트 보안 이니셔티브 Top 10. 박사 학위 논문. OWASP.

[26] LangChain... 2022... LangChain: 언어 모델 기반 애플리케이션 개발을 위한 프레임워크... <https://github.com/langchain-ai/langchain>

[27] 진세 리, 2025년, LLM 기반 에이전트에 대한 주요 패러다임 검토: 도구 사용, 계획 (RAG 포함), 피드백 학습 2025년 국제 계산 언어학 학회(COLING) Abu Dhabi, UAE, 1월 19-24일, 오웬 램바우, 류완너, 마리아나 아피디아나키, 헨드 알-칼리파, 바바라 디 에우제니, 스티븐 쇼카트 (편집) 계산 언어학 협회, 9760-9779. <https://aclanthology.org/2025.coling-main.652/>

28| LibreChat..2025..LibreChat..<https://librechat.ai>

[29] 제리 리우..2022..LlamaIndex: LLM 애플리케이션을 위한 데이터 프레임워크. https://github.com/run-llama/llama_index.

[30] 저우루 루, 토마스 홀라이스, 이즈헤 장, 베르하르트 아마이어, 평난, 펠릭스 바이, Shuang Ma, Shen Ma, 멩유 리, Guoli Yin, Zirui Wang, 그리고 Ruoming Pang.. 2024년.. ToolSandbox: LLM 도구 사용 능력 평가를 위한 Stateful, 대화형, 인터랙티브 평가 벤치마크.. CoRR abs/2408.04682 (2024).. <https://doi.org/10.48550/ARXIV.2408.04682>

[31] Baidu 지도... 2025년... Baidu 지도 MCP 서버... <https://lbs.baidu.com/faq/api?title=mecpserver/base>.

[32] Emacs MCP... 2025... Emacs MCP... <https://github.com/lizqwercott/mep.el>.

[33] Tripo3D MCP... 2025... Tripo3D MCP... <https://blender-mecp.com/>.

kmater... 2025... 도크 마스터... <https://mcp-dockmaster>

[135] mep.so...2025...MCP...so...<https://mep.so/>

이반 밀레프, 미슬라 발루노비치, 마크스틸 바더, 마틴 베체... 2025년... ToolFuzz—자동화된 에이전트. arXiv 사전 논문 arXiv:2503.04479 (2025).

[37] OpenAI... 2023... ChatGPT 플러그인... <https://openai.com/index/chatgpt-plugins/>

AI... 2023... 함수 호출... <https://platform.openai.com/docs/guides/function-calling?api-mode=resp>

[39] OpenAI... 2025년... OpenAI 에이전트 SDK - 모델 컨텍스트 프로토콜 (MCP)... <https://openai.github.io/openai-agents-python/mep/>

[40] OpenSumi..2025..OpenSumi..<https://github.com/opensumi/core>.

[41] 모델 컨텍스트 프로토콜...2024...GitHub MCP 서버...<https://github.com/modelcontextprotocol/servers/tree/main/src/>

| 모델 컨텍스트 프로토콜... 2024... 슬랙 MCP 서버... <https://github.com/modelcontextprotocol/servers/> 슬랙.

[43] Replit... 2025... Replit... <https://replit.com>.

[44] 주오칭 첸, 2024, LLM과 도구: 설문 조사, CoRR abs/2409.18807 (2024), <https://doi.org/10.48550/ARXIV.2409.18807>

[45] 즈링량 쉬, 세 가오, 시우이 첸, 유 평, 링용 얀, 하이보 쉬, 다웨이 윈, 즈민 첸, 수잔 베르베, 자오춘 렌.. 2024년.. "체인 오브 툴즈: 대규모 언어 모델은 자동 다기능 학습 도구" (CoRR abs/2405.16533, 2024). <https://doi.org/10.48550/ARXIV.2405.16533>

[46] 정량 쉬, 세 가오, 린용 얀, 유 평, 시우 첸, 주민 첸, 다웨이 윈, 수잔 베르벤, 자오춘 렌... 2025년... 야생 환경에서 도구를 학습시키기: 자동 도구 에이전트로 언어 모델 강화... 2025년 THE WEB CONFERENCE에서 발표. <https://openreview.net/forum?id=T4wMdeFEjX>

[147] 블록 (정사각형)..2025..블록 (정사각형)..<https://glama.ai/mecp/servers/atblock/square-mcp/tools/team>

[48] 스트라이프... 2025년... 스트라이프... <https://stripe.com>.

[49] 마이크로소프트 Copilot Studio..2025..Copilot Studio에 모델 컨텍스트 프로토콜 (MCP) 소개: AI 앱 및 에이전트와의 간편한 통합. <https://www.microsoft.com/en-us/microsoft-copilot/blog/copilot-studio/introducing-model-context-protocol-mcp-in-copilot-studio-simplified-integration-with-ai-apps-and-agents/>

t. 텐센트 플러그인 스토어: <https://yuanqi.tencent.com/>

[51] Apify MCP 테스터..2025년..Apify MCP 테스터..<https://apify.com/jiri.spilka/tester-mecp-client>.

[52] TheiaAI/TheiaIDE..2025..TheiaAI/TheiaIDE..https://theia-ide.org/docs/user_ai/.

[53] 쥐오겔 볼프레인, 다이크 페버, 다니엘 트룬, 오그네 란델로비치, 야콥 니콜라스 카테르..2025년..LLM 에이전트를 위한 에이전트 도구 개발..CoRR abs/2502.11705 (2025)..<https://doi.org/10.48550/ARXIV.2502.11705>

[54] 쉬리 위, 쉬유 좌, 첸 흥, 케신 흥, 미치히로 야스나가, 카이디 쉰, 바실리스 N. 요안니스, 카르틱 수비안, 주레 레스코백, 제임스 Y. 즈우..2024..AvaTaR: LLM 에이전트의 도구 사용 최적화를 위한 대조적 추론..Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, 밴쿠버, BC, 캐나다, 2024년 12월 10일 - 15일, 에미 글로버슨, 레스터 맥키, 데이애니 벨그레이브, 앙겔라 판, 울리히 파케, 야콥 M. 토메작, 첸 쉰 (편집자)..http://papers.nips.cc/paper_files/

종이/2024/해시/2db8ce969b000fe0b3fb172490c33ce8- 초록-회의.html

[55] 지heng 시, 웨상 첸, 신 구오, 웨이 헤, 이웨 딩, 보양 홍, 밍 장, 주즈 웨이, 센지 진, 엔유 주, 류 징, 샤오란 판, 샤오 왕, 류마오 셴, 유호 주, 웨이런 왕, 창하 장, 이체 쥘, 상양 류, 장유 윈, 시한 두, 롱상 웨이, 웬센 첸, 쯔장 쉰, 용연 정, 시팡 추, 셴징 황, 그리고 Tao Gui. 2023. 대규모 언어 모델 기반 에이전트의 부상과 잠재력: 개요. CoRR abs/2309.07864 (2023). <https://doi.org/10.48550/ARXIV.2309.07864>

콘스탄틴 야코블레프, 세르게이 이. 니콜렌코, 그리고 안드레이 부트... 2024년... Toolken+: LLM 도구 사용 개선 및 거부 옵션... CoRR abs/2410.12004 (2024). <https://doi.org/10.48550/ARXIV.2410.12004>

[57] 주니에 예, 시안 리, 규안 리, 카슈앙 huang, 송양 고, 룰롱 우, 쯔 장, tao gui, 쑤앙징 huang. 2024. ToolSword: 도구 학습에서 대규모 언어 모델의 안전 문제 분석 (세 단계). CoRR abs/2402.10753 (2024). <https://doi-org/10.48550/ARXIV.2402.10753>

[58] 마오 유, 판시 멩, 신운 주, 실롱 왕, 주안 마오, 린세 판, 천롱 첸, 쿤 왕, 신평 리, 용평 장, 등. 2025년. 신뢰할 수 있는 LLM 에이전트에 대한 연구: 위협 및 대응 방안. arXiv 논문 초록

[59] 시유 류안, 카타오 송, 장계 첸, 쑤 탄, 옹량 셴, 간 렌, 동성 리, 그리고 데청 양. 2024. EASYTOOL: 간결한 도구 지침을 통해 LLM 기반 에이전트 강화. CoRR abs/2401.06201 (2024). <https://doi.org/10.48550/ARXIV.2401.06201>

160} Zed... 2025... Zed - 모델 컨텍스트 프로토콜... <https://zed.dev/docs/assistant/model-context-protocol>.

[61] 밍밍 자는, 지체 왕, 유홍 난, 샤오핑 왕, 유칭 장, 그리고 젤린 양. 2022. 팀 채팅 시스템의 앱 확장 기능에서 보안 위험을 이해하기: 위험 통합. 29회 연방 및 분산 시스템 보안 심포지엄 (NDSS 2022), 샌디에이고, 캘리포니아, USA, 2022년 4월 24-28일. 인터넷 협회. <https://www.ndss-symposium.org/ndss-paper/auto-draft-262/>

양지에 좌, 신이 후, 세나 왕, 하오 유... 2024년... LLM 앱 스토어 분석: 비전 CoRR abs/2404.12737 (2024)... <https://doi.org/10.48550/ARXIV.2404.12737>